HPCC Plus: Receiver-Aware High Precision Congestion Control Study Cases and Result Figures

Project Report Draft

March 6, 2026

Contents

1 Introduction **2**

2 Standard HPCC Background **2**

3 HPCC+ Extension **3**

4 Experiment Setup **4**

5 Study Case Matrix **4**

6 Result Figures **5**

6.1 Topology Reference 5

6.2 Case 0: Full-Rate Pull Baseline 6

6.3 Cases 1 and 2: Dynamic Pull Recovery 7

6.4 Cases 3 and 4: Abrupt Throttle and Recovery 9

6.5 Cases 5 and 6: Progressive Slowdown 11

7 Interpretation Framework **13**

8 Conclusion **14**

1 Introduction

This document presents the HPCC and HPCC+ algorithms as implemented and documented in this repository, and summarizes the seven study cases currently available under the project results directory. The goal of the work is to extend fabric-aware congestion control with explicit receiver-side feedback, so that the sender can react not only to switch congestion but also to end-host receive bottlenecks.

The repository documentation establishes two layers of context:

- Standard HPCC is a rate-based congestion control algorithm that uses in-band network telemetry (INT) inserted by switches.
- HPCC+ extends this model by treating the receiver NIC ingress queue as an additional virtual hop in the INT feedback chain.

This is useful when the network fabric is lightly loaded but the receiver is temporarily unable to pull packets fast enough. In that situation, standard HPCC may only observe the problem indirectly, while HPCC+ turns the receiver backlog into a first-class congestion signal.

2 Standard HPCC Background

According to the repository documentation, HPCC uses per-hop INT feedback to estimate link utilization with much higher precision than binary signals such as ECN. Each switch contributes four key fields to the INT record of a packet:

- a timestamp,
- transmitted byte counter,
- current queue length,
- and the line rate of the output port.

From two consecutive feedback samples, the sender estimates the utilization of each hop. The control law described in the repository can be summarized as

$$u_h = \frac{\mathrm{TxRate}_h}{C_h} + \frac{Q_h}{C_h \cdot RTT_{base}}$$

where TxRate_h is the measured transmission rate at hop h, C_h is the line rate of that hop, Q_h is the queue contribution, and RTT_{base} is the base round-trip time used to normalize the queue term.

For a path with multiple INT-enabled hops, standard HPCC identifies the largest instantaneous utilization value and uses it as the bottleneck estimate:

$$U_{\max} = \max_h u_h$$

That estimate is then smoothed with an RTT-weighted filter before the sender updates its rate. The repository documentation describes the rate update in the form

$$R_{new} = \frac{R_{current}}{U_{norm}} + R_{AI}$$

where $U_{norm} = U_{\max}/\eta$ and η is the target utilization, typically configured as 0.95. Under low load, HPCC can remain in additive increase mode for a configurable number of stages, then switch into a more aggressive multiplicative regime.

The pacing model is equally important. HPCC is rate-based first, and the congestion window acts as a secondary bound. The sender transmits each packet with an inter-packet gap derived from packet size and target rate:

$$\Delta T_{gap} = \frac{\text{PacketSize (bits)}}{R \text{ (bits/s)}}$$

This design aims to keep queues shallow while still driving the fabric close to its target utilization.

3 HPCC+ Extension

The HPCC+ documentation in this repository describes a receiver-aware extension that adds a host INT hop at the receiver side. Instead of limiting telemetry to switches, HPCC+ inserts one extra hop into the INT list before the ACK is generated. That host hop carries:

- the receiver timestamp,
- the cumulative number of bytes drained from the receiver NIC,
- the current receiver ingress queue occupancy,
- and the receiver NIC line rate.

The sender processes all switch hops exactly as in standard HPCC, and then separately processes the final host hop. The delivered rate at the receiver is estimated as

$$R_{delivered} = \frac{(rxBytes^{new} - rxBytes^{old}) \cdot 8}{\tau_{host}}$$

where τ_{host} is the time between the two host INT samples.

HPCC+ then tracks an effective host capacity C_{host} rather than blindly using the physical NIC line rate. This matters because a CPU-limited or software-limited receiver may drain traffic at a much lower rate than the line can support. When the receiver queue is non-empty, the sender treats the observed delivered rate as an estimate of the real bottleneck and updates C_{host} with an EWMA. When the receiver queue is empty, C_{host} can increase additively toward the physical rate.

The host-side utilization term described in the documentation is

$$u_{host} = \begin{cases} 1 + \dfrac{qlen_{rx}}{C_{host} \cdot RTT_{base}} & \text{if } qlen_{rx} > 0 \\ \dfrac{R_{delivered}}{C_{host}} & \text{if } qlen_{rx} = 0 \end{cases}$$

The final congestion signal becomes

$$U_{final} = \max(\hat{u}_{switch}, u_{host})$$

This is the key conceptual difference between HPCC and HPCC+: HPCC observes only the network fabric, while HPCC+ observes both the network fabric and the receiver's ability to consume data.

4 Experiment Setup

The study cases in this repository share a simple but useful topology: two senders feed a single receiver through a switch. The topology file used by the cases defines three 10 Gb/s links with 0.01 ms link delay. The flow file defines two long-lived sender-to-receiver flows:

- sender 0 to receiver 2,
- sender 1 to receiver 2.

This setup isolates the effect of receiver pulling rate changes. Because the two senders share the same destination, the experiment is a direct test of how each congestion control mode reacts when the receiver becomes the limiting resource.

Across the cases, the main common settings are:

- packet payload size of 1000 bytes,
- 10 Gb/s links,
- fast reaction enabled,
- target utilization $U_{target} = 0.95$,
- additive increase step $R_{AI} = 50$ Mb/s,
- per-packet INT feedback enabled.

Cases using standard HPCC set `CC_MODE = 3`. Cases using HPCC+ set `CC_MODE = 11`. The experimental variable is the receiver pull schedule.

5 Study Case Matrix

Table 1 summarizes the seven available study cases.

Case	Mode	Receiver Pull Schedule	Purpose
0	HPCC	Immediate drain, effectively full-rate pull	Baseline reference with no receiver bottleneck
1	HPCC	Start at 10%, then 5%, 20%, 50%, 100% at 1.0, 1.2, 1.4, 1.6 s	Dynamic receiver recovery under baseline HPCC
2	HPCC+	Same as Case 1	Dynamic receiver recovery with explicit host feedback
3	HPCC	100%, 5%, 20%, 50%, 100% at 1.0, 1.2, 1.4, 1.6, 1.8 s	Abrupt throttle and recovery under HPCC
4	HPCC+	Same as Case 3	Abrupt throttle and recovery under HPCC+
5	HPCC	100%, 90%, 70%, 50%, 10% at 1.0, 1.2, 1.4, 1.6, 1.8 s	Progressive receiver slowdown under HPCC
6	HPCC+	Same as Case 5	Progressive receiver slowdown under HPCC+

Table 1: Study case summary derived from the repository configuration files.

6 Result Figures

Each case directory contains a consistent set of figures: topology, flow completion time, receiver buffer occupancy, PFC pause events, switch queue depth CDF, ACK analysis, CWND and RTT analysis, switch throughput, and a dashboard plot. In this report, the dashboard and FCT figures are intentionally omitted from the visual presentation. Instead, the report includes all of the remaining static plots, while FCT is stated as text using the generated data files where they are available.

6.1 Topology Reference

The same logical topology is used throughout the study cases, so one topology plot is sufficient as a structural reference.

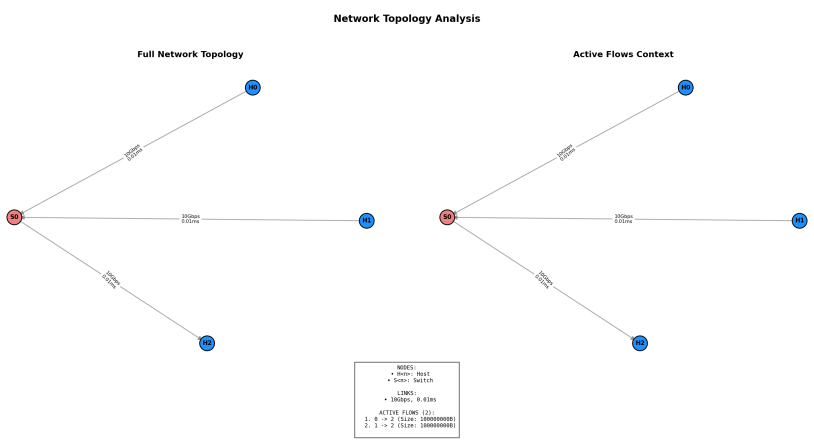
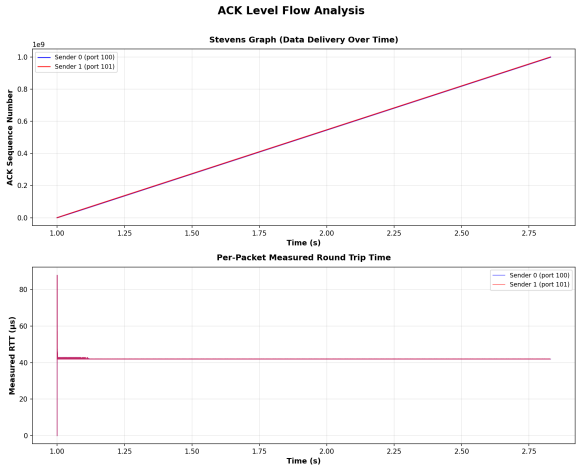


Figure 1: Topology visualization for the two-sender, one-receiver study setup.

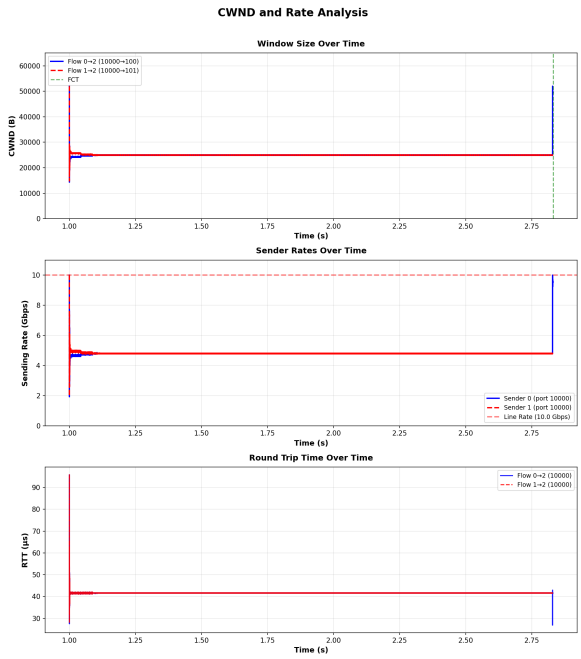
6.2 Case 0: Full-Rate Pull Baseline

Case 0 is the control experiment. The receiver drains immediately, so the dominant congestion signal should come from the network path rather than the end host. This case provides the reference behavior against which the dynamic-pull cases can be interpreted.

FCT summary: the baseline FCT plot exists in the copied assets, but the corresponding raw FCT data file is not present under the case-local data directory in the current workspace snapshot. For that reason, this report omits the FCT figure and does not state a numerical baseline FCT summary for Case 0.

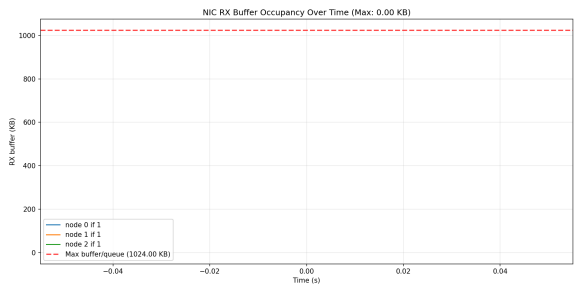


(a) Case 0 ACK analysis

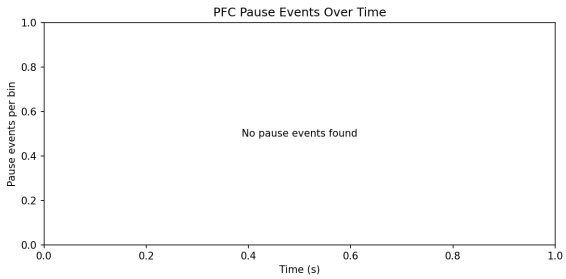


(b) Case 0 CWND and RTT analysis

Figure 2: Baseline ACK-side and rate-window behavior under full-rate pull.

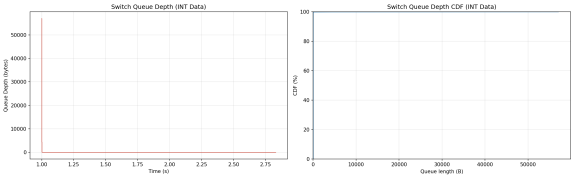


(a) Case 0 receiver buffer

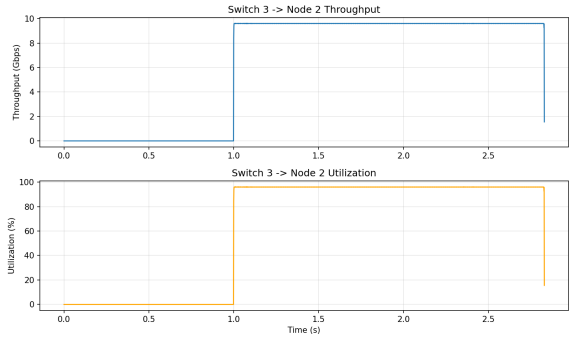


(b) Case 0 PFC pause events

Figure 3: Receiver-side buffering and pause behavior for the baseline case.



(a) Case 0 switch queue depth CDF



(b) Case 0 switch throughput

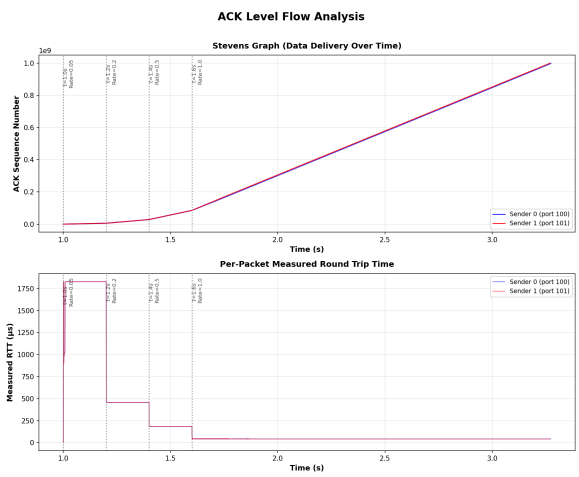
Figure 4: Switch queue occupancy and throughput under full-rate pull.

6.3 Cases 1 and 2: Dynamic Pull Recovery

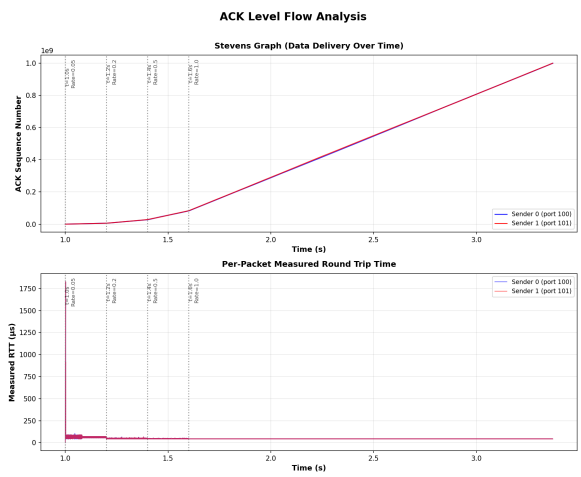
Cases 1 and 2 apply the same receiver schedule but use different congestion control modes. The receiver starts at 10% of line rate and then progresses through 5%, 20%, 50%, and finally 100%. This pair is a direct comparison of whether explicit receiver feedback helps the sender track recovery more cleanly.

In standard HPCC, receiver congestion can only be inferred indirectly from downstream consequences such as queue growth and delayed ACK behavior. In HPCC+, the host INT hop converts the receiver backlog into an explicit utilization signal. For that reason, this pair is central to the HPCC+ argument.

FCT summary: both cases contain two completed 1 GB flows. Case 1 records mean FCT of 2.272 s, median FCT of 2.270 s, and maximum FCT of 2.274 s. Case 2 records mean FCT of 2.372 s, median FCT of 2.372 s, and maximum FCT of 2.372 s.

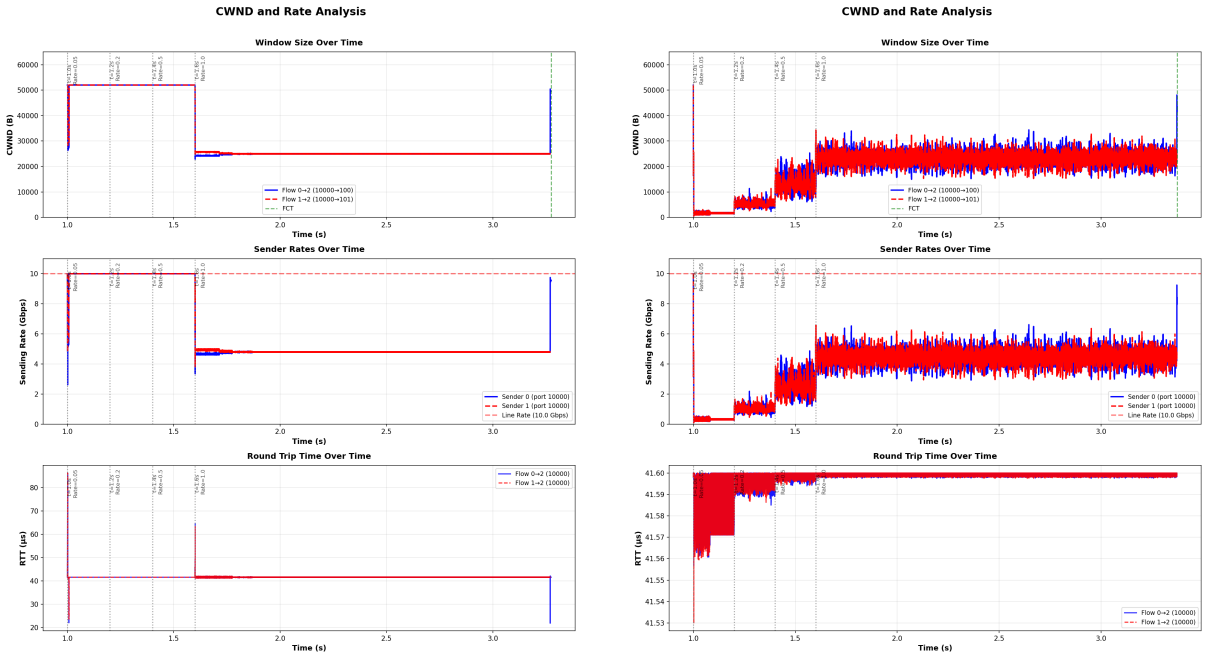


(a) Case 1: HPCC ACK analysis



(b) Case 2: HPCC+ ACK analysis

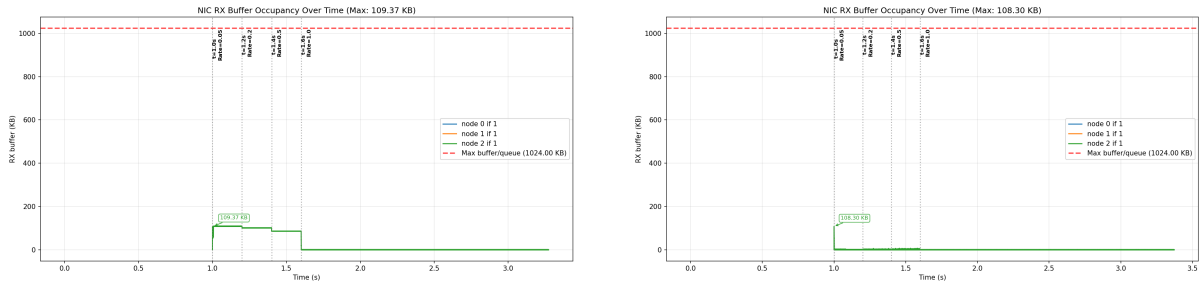
Figure 5: ACK-side comparison for the dynamic recovery study.



(a) Case 1: HPCC CWND and RTT analysis

(b) Case 2: HPCC+ CWND and RTT analysis

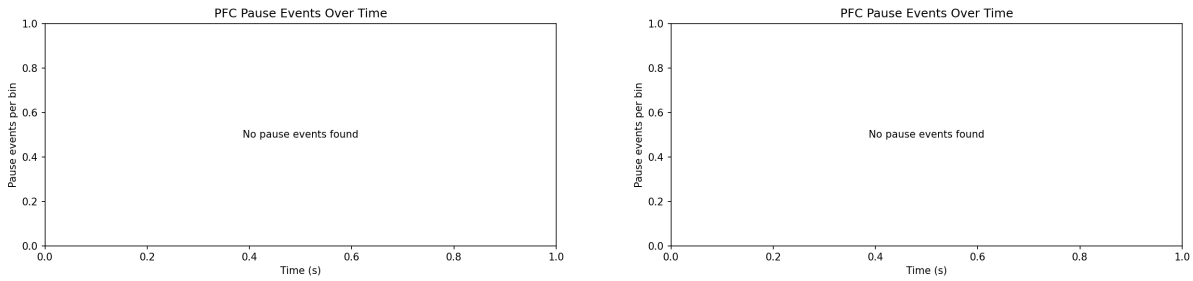
Figure 6: Rate-window comparison for the dynamic recovery study.



(a) Case 1: HPCC receiver buffer

(b) Case 2: HPCC+ receiver buffer

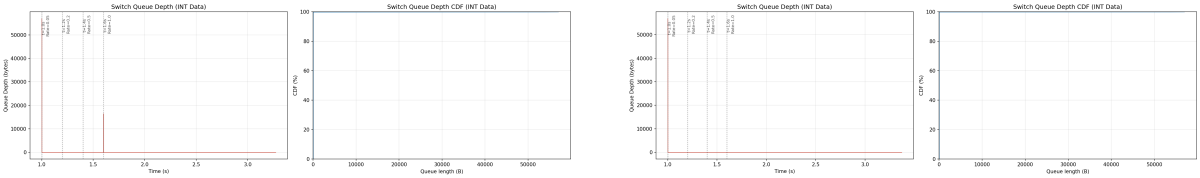
Figure 7: Receiver buffer occupancy under the same recovery schedule.



(a) Case 1: HPCC PFC pause events

(b) Case 2: HPCC+ PFC pause events

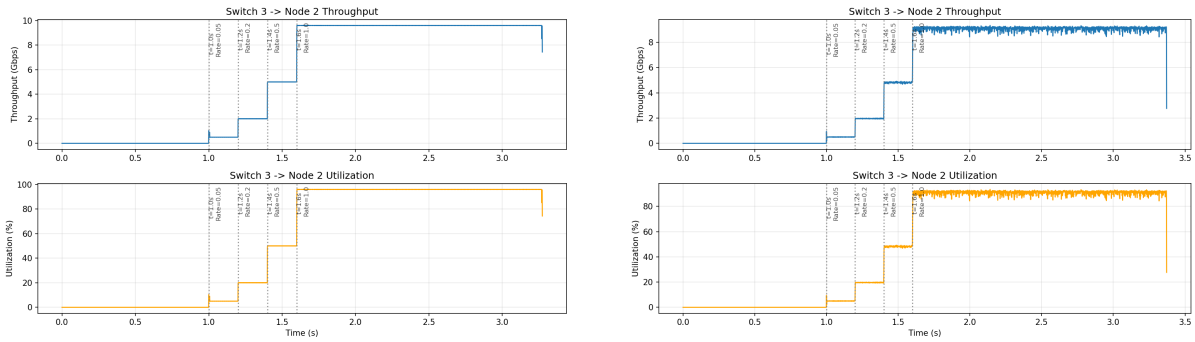
Figure 8: Pause-event comparison for the dynamic recovery study.



(a) Case 1: HPCC switch queue depth CDF

(b) Case 2: HPCC+ switch queue depth CDF

Figure 9: Switch queue depth comparison for the dynamic recovery study.



(a) Case 1: HPCC switch throughput

(b) Case 2: HPCC+ switch throughput

Figure 10: Switch throughput comparison for the dynamic recovery study.

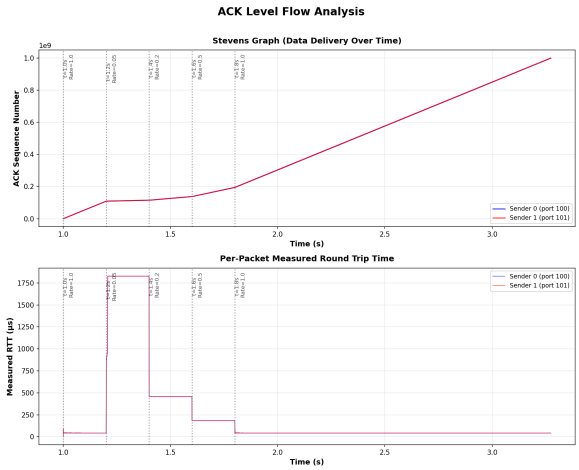
6.4 Cases 3 and 4: Abrupt Throttle and Recovery

Cases 3 and 4 begin at full receiver capacity, sharply drop to 5%, then recover in stages to 20%, 50%, and 100%. This is a stronger stress test than Cases 1 and 2 because the path begins uncongested, then experiences a sudden end-host bottleneck.

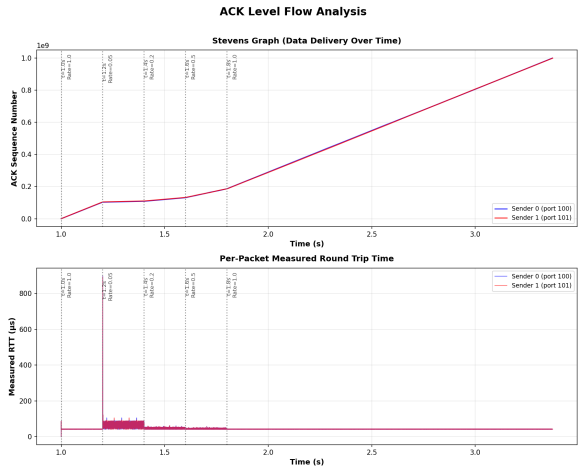
This family is useful for evaluating two questions:

- how quickly the sender reacts to the abrupt drop, and
- how gracefully it reclaims bandwidth after the receiver recovers.

FCT summary: both cases contain two completed 1 GB flows. Case 3 records mean FCT of 2.273 s, median FCT of 2.273 s, and maximum FCT of 2.274 s. Case 4 records mean FCT of 2.373 s, median FCT of 2.372 s, and maximum FCT of 2.374 s.

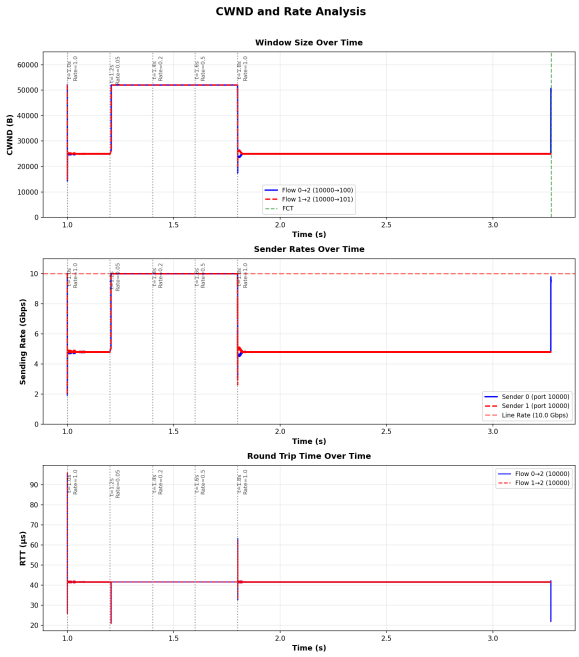


(a) Case 3: HPCC ACK analysis

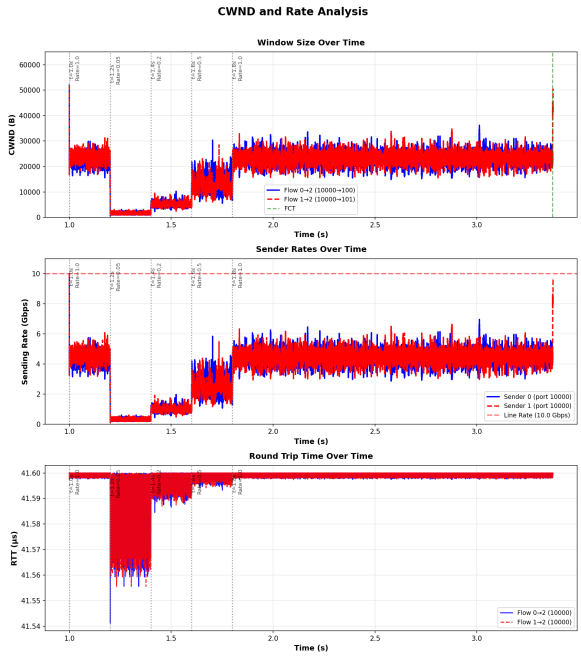


(b) Case 4: HPCC+ ACK analysis

Figure 11: ACK-side comparison for the abrupt throttle and recovery study.

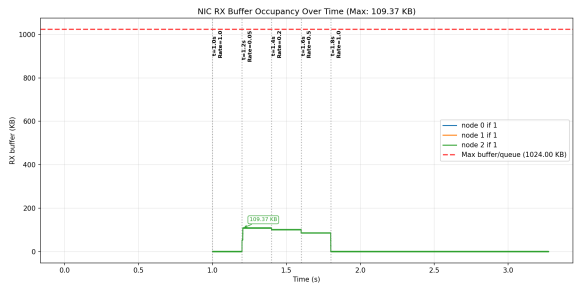


(a) Case 3: HPCC CWND and RTT analysis

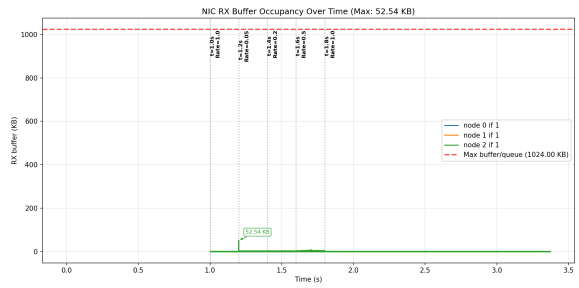


(b) Case 4: HPCC+ CWND and RTT analysis

Figure 12: Rate-window comparison for the abrupt throttle and recovery study.

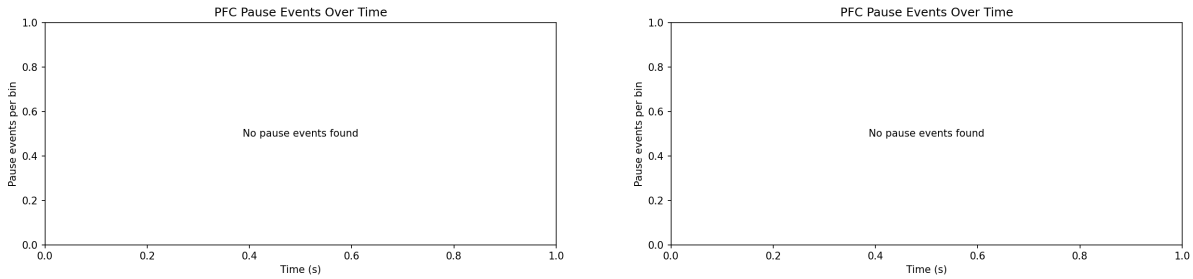


(a) Case 3: HPCC receiver buffer



(b) Case 4: HPCC+ receiver buffer

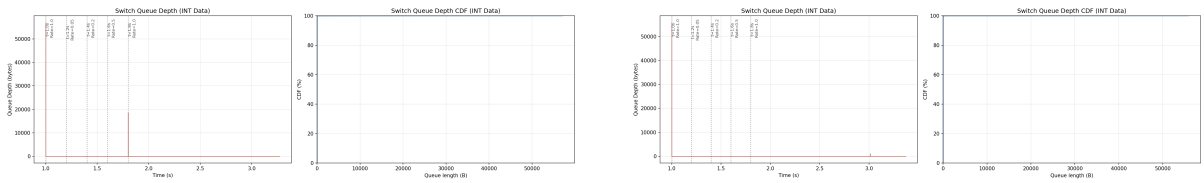
Figure 13: Receiver buffer comparison for the abrupt throttle and recovery schedule.



(a) Case 3: HPCC PFC pause events

(b) Case 4: HPCC+ PFC pause events

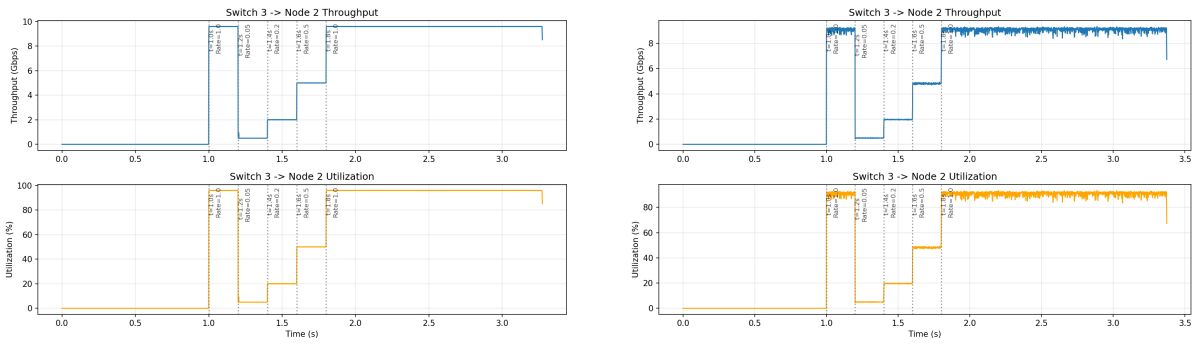
Figure 14: Pause-event comparison for the abrupt throttle and recovery study.



(a) Case 3: HPCC switch queue depth CDF

(b) Case 4: HPCC+ switch queue depth CDF

Figure 15: Switch queue depth comparison for the abrupt throttle and recovery study.



(a) Case 3: HPCC switch throughput

(b) Case 4: HPCC+ switch throughput

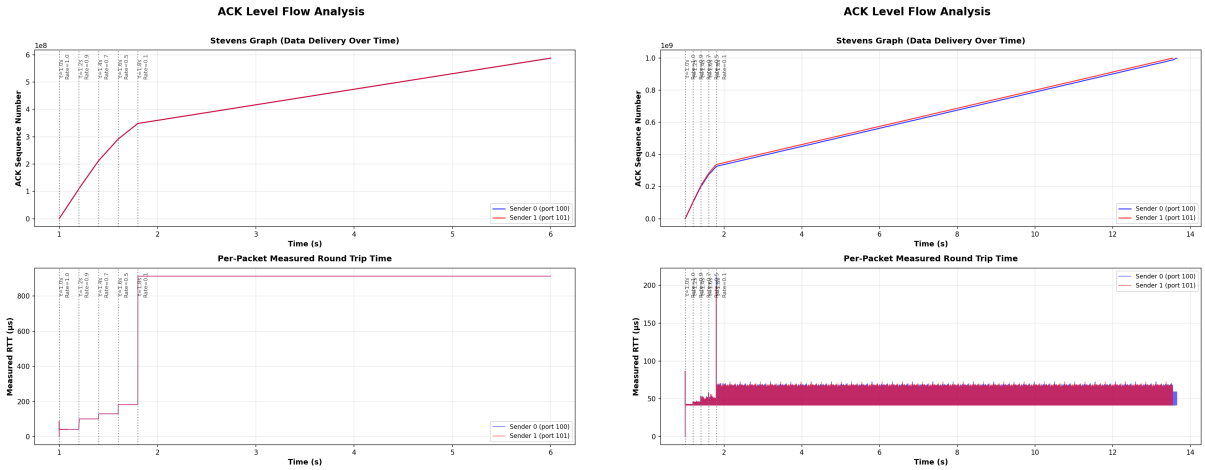
Figure 16: Switch throughput comparison for the abrupt throttle and recovery study.

6.5 Cases 5 and 6: Progressive Slowdown

Cases 5 and 6 start at full receiver capacity and then progressively degrade to 90%, 70%, 50%, and finally 10%. This pattern is different from the previous pair because it models a sustained deterioration rather than a recovery process.

For presentation purposes, these cases are valuable because they show how the control loop behaves when the end host becomes steadily worse over time. HPCC+ is specifically designed for this type of situation: the host hop should keep the sender aligned with the receiver's actual service rate instead of assuming the network remains the only bottleneck.

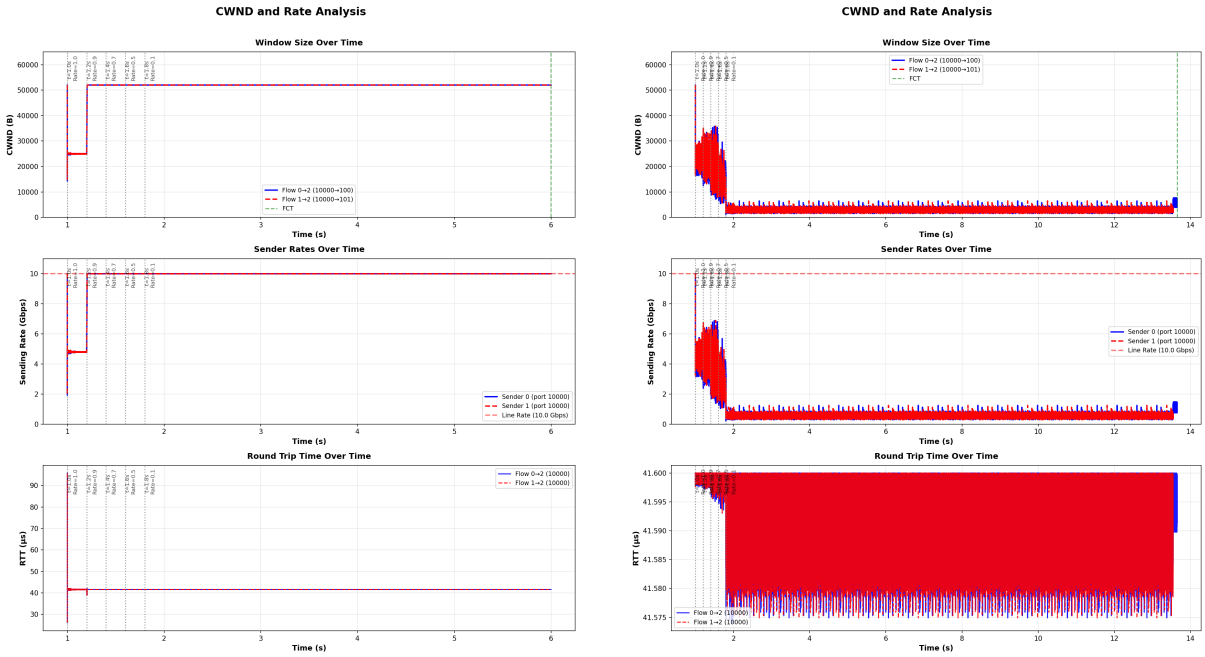
FCT summary: Case 5 has an empty FCT data file in the current results set, so no numerical FCT summary can be stated for that run. Case 6 contains two completed 1 GB flows with mean FCT of 12.595 s, median FCT of 12.543 s, and maximum FCT of 12.647 s.



(a) Case 5: HPCC ACK analysis

(b) Case 6: HPCC+ ACK analysis

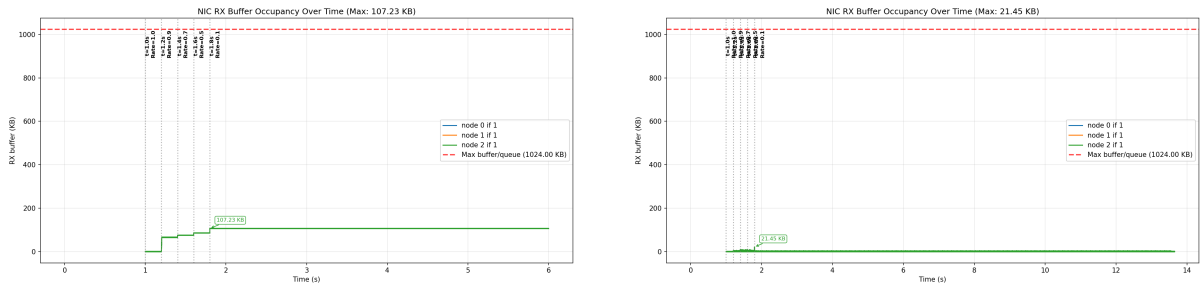
Figure 17: ACK-side comparison for the progressive slowdown study.



(a) Case 5: HPCC CWND and RTT analysis

(b) Case 6: HPCC+ CWND and RTT analysis

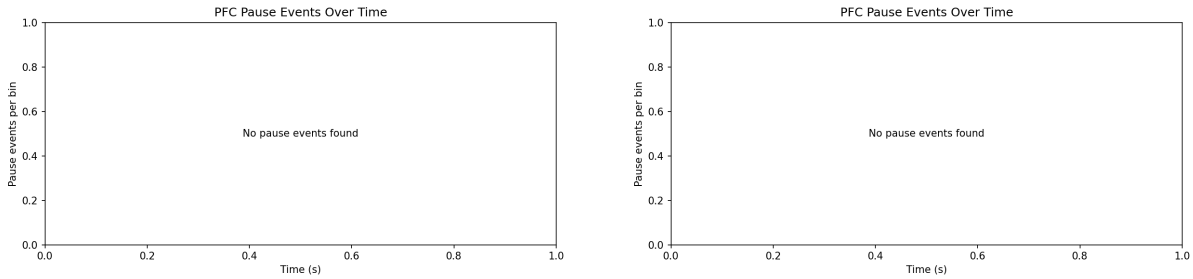
Figure 18: Rate-window comparison for the progressive slowdown study.



(a) Case 5: HPCC receiver buffer

(b) Case 6: HPCC+ receiver buffer

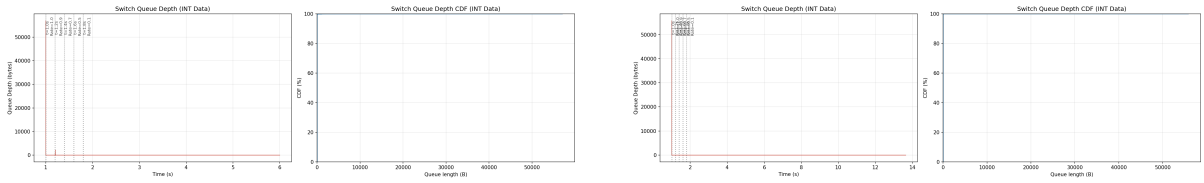
Figure 19: Receiver buffer comparison for the progressive slowdown schedule.



(a) Case 5: HPCC PFC pause events

(b) Case 6: HPCC+ PFC pause events

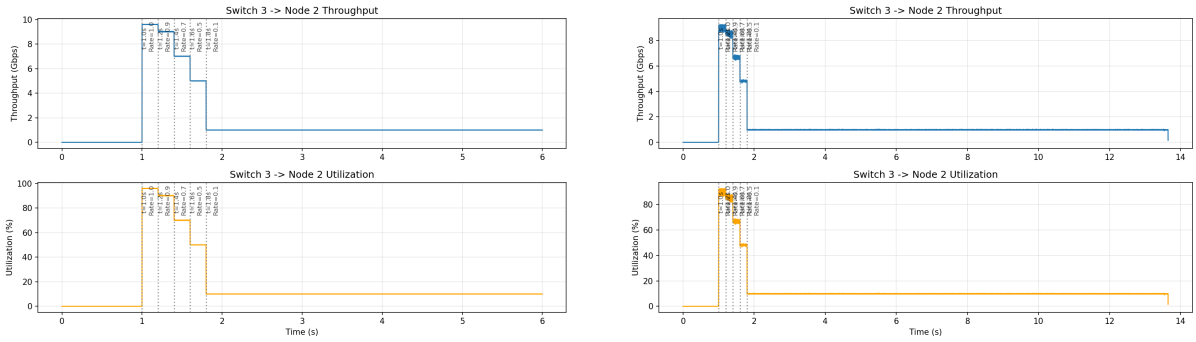
Figure 20: Pause-event comparison for the progressive slowdown study.



(a) Case 5: HPCC switch queue depth CDF

(b) Case 6: HPCC+ switch queue depth CDF

Figure 21: Switch queue depth comparison for the progressive slowdown study.



(a) Case 5: HPCC switch throughput

(b) Case 6: HPCC+ switch throughput

Figure 22: Switch throughput comparison for the progressive slowdown study.

7 Interpretation Framework

Because the purpose of this document is to present the study cases and bundle the existing figures into one LaTeX report, the most defensible way to read the plots is along the following dimensions:

- **Rate tracking:** does the sending rate align with the receiver's available pull rate after a schedule change?
- **Queue containment:** does the receiver buffer remain controlled when the receiver becomes the bottleneck?
- **Fabric side effects:** do switch queue depth, throughput, and PFC plots remain stable while the end host is changing state?

- **Application impact:** how do the FCT plots shift between HPCC and HPCC+ under the same schedule?

The expected advantage of HPCC+ is not that it ignores the network, but that it unifies both bottleneck classes. If congestion comes from a switch, HPCC+ still behaves like HPCC. If congestion comes from the receiver, HPCC+ can react to it directly through the host INT hop instead of waiting for indirect secondary signals.

8 Conclusion

The repository combines a standard HPCC simulator with an HPCC+ extension aimed at explicit receiver-side congestion awareness. The study cases are organized around a clean comparison pattern:

- one baseline full-rate case,
- one HPCC versus HPCC+ pair for dynamic recovery,
- one HPCC versus HPCC+ pair for abrupt throttle and recovery,
- and one HPCC versus HPCC+ pair for progressive slowdown.

This report pulls those experiments into a presentation-ready LaTeX structure. It explains the control logic of HPCC, the host-feedback extension introduced by HPCC+, and embeds the result figures required to discuss the behavior of both algorithms across all seven scenarios.

In a later revision, this document can be extended with manually extracted quantitative observations from the plots, such as approximate settling times, peak buffer occupancy, and relative FCT shifts between HPCC and HPCC+.